

GIT & GITHUB

Foundations for the practical workshop

Local files

Your working folder



Git

Local history



GitHub

Backup & collaboration

Why do we need version control?

1

No history

Overwrite a file and the previous version is gone for good.

2

No safety net

One bad edit and there's no way back to what worked.

3

No collaboration

Emailing zip files and merging changes by hand doesn't scale.

Git solves all three: it tracks every change, lets you undo safely, and gives every collaborator a full, synchronized copy of the project.

Git vs GitHub

Git

A version control system

- Runs on your computer
- Tracks history locally, offline
- Command-line tool (git init, add, commit...)
- Works even with no internet connection

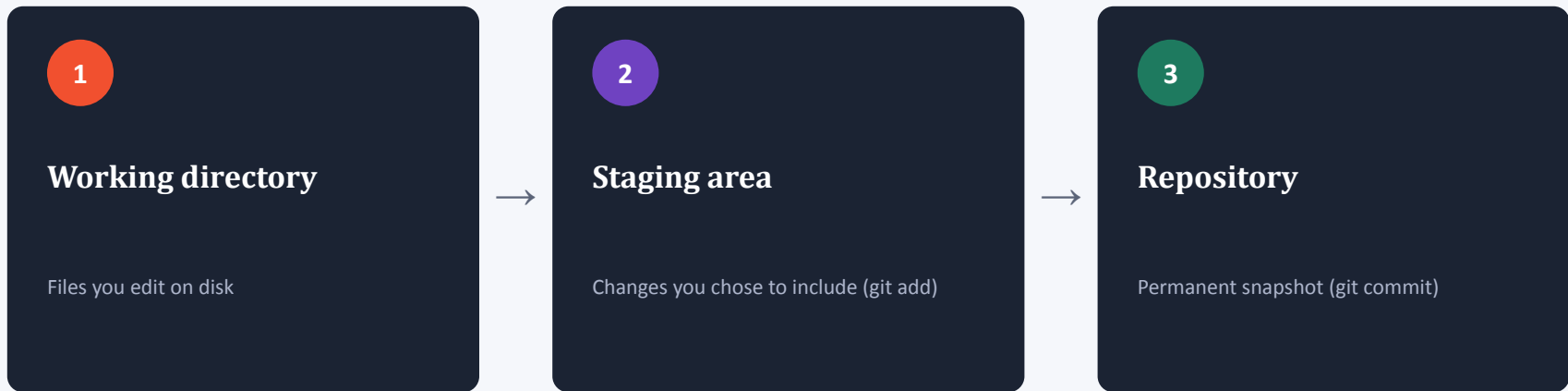
GitHub

A hosting & collaboration platform

- Lives in the cloud
- Hosts a remote copy of your repository
- Adds pull requests, issues, pages, teams
- Requires Git underneath to push/pull

Three areas, every single time

Every change you make travels through the same three stages before it's safely recorded.



Core concepts you'll use constantly

repository

A folder tracked by Git; keeps the full history of every file inside it.

commit

A saved snapshot of your project, with a message and a unique hash.

staging area

A holding zone where you pick exactly what the next commit will include.

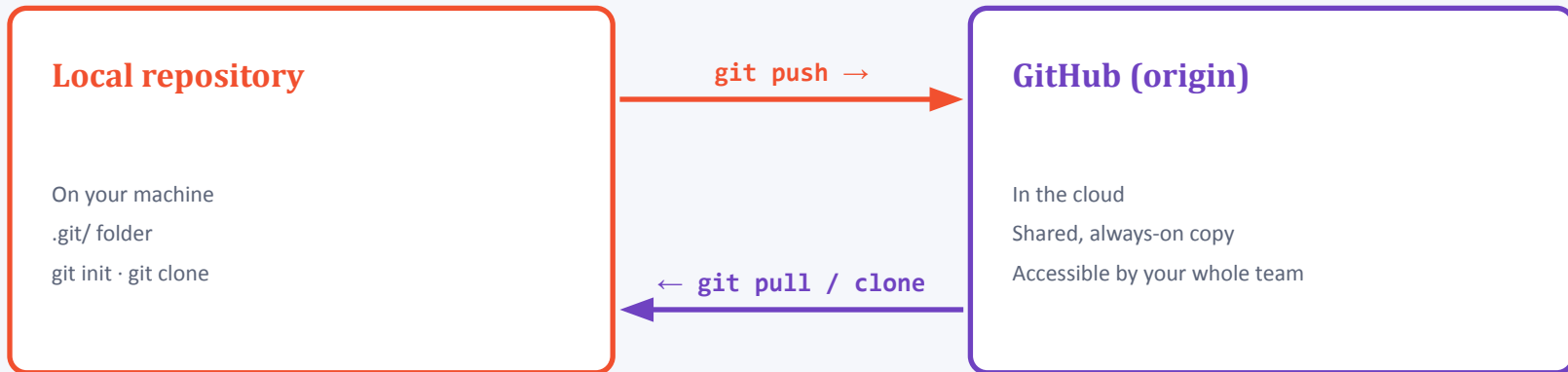
HEAD

A pointer to the commit you currently have checked out.

remote

A version of your repository hosted elsewhere — usually GitHub, named origin.

Connecting local Git to GitHub



Push vs pull, in one picture

git push

Send your local commits up to GitHub.

Local repository



GitHub

Used after: git add + git commit

git pull

Bring GitHub's commits down to your machine.

GitHub



Local repository

Used to sync teammates' work

Branching: work without breaking main

A branch is a parallel line of history. Experiment freely on it — main stays untouched until you decide to merge.



Diverge, then merge back

1

Create the branch

```
git checkout -b new-feature
```

2

Commit on both

main and new-feature evolve
separately

3

Visualize

```
git log --oneline --graph  
--all
```

4

Merge back

```
git checkout main · git merge  
new-feature
```

Once merged, main contains every commit from both lines of work — nothing is lost.

Merge conflicts

Git merges automatically whenever it can. A conflict only happens when both branches changed the exact same lines.

Conflict markers

```
<<<<<<< HEAD
recipe: ratatouille
=====
recipe: pasta
>>>>>>> new-feature
```

How to resolve it

- Open the file and locate the markers
- Keep the content you want, delete the markers
- Stage the resolved file: `git add <file>`
- Finish it: `git commit`

What Git should never track

A .gitignore file tells Git to skip certain files completely — they never appear in git status, and git add . can't pick them up by accident.

.gitignore

```
*.log  
__pycache__/  
.env  
config.env  
node_modules/
```

Never commit

- Passwords & API keys
- Database credentials (.env)
- Personal access tokens
- Anything already exposed must be rotated — deleting it later isn't enough

The cycle you'll repeat every day



Windows note: run this from Git Bash or the VS Code integrated terminal (Git Bash profile) — same commands, no apt needed.

Key takeaways

You now have the vocabulary for everything we'll practice.

- Git tracks history locally; GitHub hosts and shares it
- Every change flows: working directory → staging → repository
- push sends work up, pull brings work down
- Branches let you experiment without breaking main
- Conflicts are normal — resolve, add, commit
- .gitignore keeps secrets out of your history